

CAA 25-26

Lab #1

17-09-2025

The goal of this lab is to use the **Hashicorp Vault** software (<https://www.vaultproject.io>) to manage secrets in a company infrastructure. Vault is heavily used in industry but became closed source in 2023. Hence, we will work with the open source version **openBao** (openbao.org). However, we will often use the name “Vault” to name the software as this is the name known in the industry. Vault is a software that provides the management and storage of secret information, allows to do precise access control, and has further advanced functionalities like certificate and one-time passwords generation. In particular, Vault is great to **centralize** secrets in a company. It allows easy access revocation as well as auditing (who accessed what and when).

1 Submission

You are required to submit

- A script setting up a “dev” Vault instance for the scenario with the required groups and users.
- Two software (preferably in Python): one allowing to send/receive encrypted messages, one polling Vault to see if there are new messages
- A small report in which you simply answer the questions in each section.

2 Installation and Launch

First, install openBao: <https://openbao.org/docs/install/>. If you prefer doing everything with Hashicorp Vault, this is fine. Then launch a server in “dev” mode. Beware that this mode is **insecure** and doesn’t allow any persistence. Although this is also not a good practice, we will do all the setup using the “root token”. Read about the main concepts of Vault here: <https://openbao.org/docs/concepts/>

- 2.1. What are tokens used for and how do they look like in OpenBao?
- 2.2. How do you set up what are the capabilities of a token?
- 2.3. What is sealing and unsealing and what is it used for?

3 Main App Description

The goal of our main app is to have confidential discussion groups. Users can be members of these groups and send encrypted messages on them (like a forum). The encryption keys are stored on the Vault. The encrypted messages and metadata will also be stored there. More precisely, we will use the **transit** engine to encrypt message (<https://openbao.org/docs/secrets/transit/>) and the kv-v2 store to store the ciphertexts (<https://openbao.org/docs/secrets/kv/kv-v2/>).

- Activate both engines inside your vault.
- 3.1. Why isn’t it a great idea to store the ciphertexts on Vault? (We do it to avoid having to setup a database).

4 Creating Groups

We are going to create discussion groups manually (Vault allows a more complex group management using the Identity and Group feature but we will ignore it for this lab). In this lab, we are going to create two discussion groups: **Financial** and **IT**. We will also have three users:

- **ceo** which has access to both groups,
- **bob** who has access only to **IT**,
- and **secretary** which has access only to **Financial**.

Tasks:

- For each group, create a key to use encryption-as-a-service. Make sure that **Financial** uses an AES256-GCM key and that **IT** uses XChach20-Poly1305. Make the keys rotate every 1h.
- For each group, create a policy file that we will assign to users and that allows them to encrypt/decrypt using the group key. We will also store the ciphertexts in a kv-v2 store that has the name of the group. Users should be able to read and write there. The path is `<kv>/data/ciphertexts/<groupName>`, where `<kv>` is the mount point of your kv store.
- Finally, users should be able to login with a username and password (<https://openbao.org/docs/auth/userpass/>). The logins are **ceo:ceo1**, **bob:bob1**, **secretary:secretary1**. Create the users and give them the correct access rights.
- (optional) You can test what a token can do at a specific endpoint with the command `bao capabilities <token> <path>`

Questions:

- 4.1. What is key rotation and what is it used for?
- 4.2. How does Vault make sure one can decrypt an old message when keys are rotated?
- 4.3. What do you think of having a key rotation every 1h?
- 4.4. What capabilities did you give for the transit service and why? Provide the policy file in your report.
- 4.5. What capabilities did you give for the kv service and why? Provide the policy file in your report.

5 Main App Development

For the Python code to communicate with Bao/Vault, use the **hvac** library. If you prefer Go, Java, C#, you can also do it in these languages. The following example <https://github.com/hashicorp/vault-examples/blob/main/examples/quick-start/python/example.py> will help.

Write a Python app using the hvac library that

- Logs the user in with a username and password
- Asks the user for a group name
- Asks the user if he wants to send or receive a message in this group
- If send: asks for a content and a title and stores the encrypted message in the kv store for that group.
- If receive: asks only for the title and prints the decrypted payload.

Your program can crash in case of failure.

- 5.1. Show in your report that bob cannot send or receive data from the finance group.
- 5.2. Show in your report that your app is working properly.

6 Watcher App

Now we want a watcher app that reads every 5s on **all** groups to know whether there is a new encrypted message. This watcher app should not have other access rights. The watcher should login using an **app token** that is given to the program as an **environment variable**. When there is a new message, you should print its group, its title and the username of the sender.

- 6.1. Show in your report that your app token cannot encrypt or decrypt data.
- 6.2. Show in your report that your app is working properly.